

Milestone H

Acceptance Test Cases | Test Execution Results |
Identified Issues and Improvements

Embedded Systems - Face Detection Game Project

SDU - Software Engineering

May 2026

1. Test Scope and Targets

Acceptance testing validates that the Face Detection Game meets its functional requirements from the player's perspective. Tests were executed on a PYNQ Z2 board running the full system: HDMI output via camera.py, MediaPipe face detection backend, and four physical push-buttons for input.

1.1 Testing Targets

- Game state transitions (IDLE -> RUNNING -> PAUSED -> END)
- Face detection accuracy and robustness across angles and lighting
- Round scoring and strike logic
- Difficulty scaling (timer reduction as score increases)
- Hardware button responsiveness and control mapping
- HUD accuracy (score, strikes, face count, timer bar)
- System performance (frame rate on target hardware)

1.2 Test Environment

Property	Value
Hardware	PYNQ Z2 (Xilinx Zynq-7020)
OS	PYNQ Linux (Ubuntu-based)
Python	3.8
OpenCV	4.5.4 (apt-installed)
Detection backend	MediaPipe (primary)
Camera	USB webcam, 640x480
Display	HDMI monitor via base overlay
Input	4 physical push-buttons (BTN0-BTN3)

1.3 Test Case Summary

ID	Title	Status
TC-01	Game start from IDLE state	PASS
TC-02	Face detection -- frontal face	PASS
TC-03	Face detection -- angled face	FAIL
TC-04	Round scoring when target met	PASS
TC-05	Strike added when target not met	PASS
TC-06	Game over at 3 strikes	PASS
TC-07	Pause and resume	FAIL
TC-08	Face target adjustment via buttons	FAIL
TC-09	Difficulty scaling with score	PASS
TC-10	Frame rate on PYNQ Z2	FAIL
TC-11	HUD display accuracy	PASS

TC-12	Restart from END state	PASS
-------	------------------------	------

2. Acceptance Test Cases and Results

TC-01 -- Game start from IDLE state [PASS]

Precondition

Board powered on. Game in IDLE state. HDMI monitor connected.

TC-02 -- Face detection -- frontal face [PASS]

Precondition

Game RUNNING. max_faces=1. Room lighting adequate (>100 lux).

TC-03 -- Face detection -- angled face [FAIL]

Precondition

Game RUNNING. max_faces=1. Face detected frontally (TC-02 passed).

TC-04 -- Round scoring when face target is met [PASS]

Precondition

Game RUNNING. max_faces=1. Score=0.

TC-05 -- Strike added when face target is not met [PASS]

Precondition

Game RUNNING. max_faces=2. Only 1 person available.

TC-06 -- Game over at 3 strikes [PASS]**Precondition**

Game RUNNING. strikes=2. max_faces=2.

TC-07 -- Pause and resume [FAIL]**Precondition**

Game RUNNING. Timer counting down.

TC-08 -- Face target adjustment via buttons [FAIL]**Precondition**

Game in any state.

TC-09 -- Difficulty scaling with score [PASS]**Precondition**

Game RUNNING. Score=0.

TC-10 -- Frame rate on PYNQ Z2 [FAIL]**Precondition**

Game RUNNING. MediaPipe detection backend active.

TC-11 -- HUD display accuracy [PASS]**Precondition**

Game RUNNING. score=3, strikes=1, max_faces=2, current_faces=1.

TC-12 -- Restart from END state [PASS]**Precondition**

Game in END state. Final score displayed.

3. Identified Issues and Improvements

3.1 Issue Log

ID	Severity	Description	Failing TCs
ISS-01	Critical	Frame rate 0.5-0.8 FPS on PYNQ Z2. MediaPipe inference takes ~1.2s per frame on Cortex-A9.	TC-10
ISS-02	High	Buttons must be held (~0.5s) to register. Short taps are missed by the 10ms poll interval.	TC-07, TC-08, TC-12
ISS-03	Medium	Face detection fails at angles > ~20 degrees. Only strict frontal faces reliably detected.	TC-03
ISS-04	Low	When button held, face target increments rapidly (no repeat delay), making it difficult to track.	TC-08

3.2 Issue Detail and Proposed Fixes

ISS-01 -- Frame Rate (Critical)

Root cause: MediaPipe's BlazeFace TFLite model is compiled for ARMv7 without NEON SIMD acceleration in the pip-distributed wheel for this Python version. The PYNQ Z2 has a dual-core Cortex-A9 at 666 MHz, which is significantly slower than a Raspberry Pi 4 (Cortex-A72 at 1.5 GHz).

- Switch to Haar Cascade backend: 10-25ms per frame vs 1200ms. Achieves ~30 FPS at the cost of detection quality.
- Detection frame skipping: run detector every 3rd frame, reuse previous result. Triples effective throughput with no impact on game timing.
- Reduce input resolution to 320x240 before passing to detector. Saves the internal resize step inside TFLite.
- Compile OpenCV with NEON support from source to accelerate Haar cascade further.

ISS-02 -- Button Hold Required (High)

Root cause: PYNQ Z2 buttons are polled inside the main game loop at ~10ms intervals. At 0.5-0.8 FPS the actual poll interval is 1.2-2.0 seconds -- far too slow to catch a brief tap. The edge detection compares current vs previous sample, but if a tap happens between two samples it is never seen.

- Use PYNQ button interrupts instead of polling: `base.buttons[i].wait_for_value(1)` in a dedicated daemon thread. This detects presses regardless of frame rate.
- As an immediate workaround: switch to Haar Cascade (ISS-01 fix) to raise FPS, which reduces the poll interval and makes short taps detectable.

```
# Interrupt-based fix (daemon thread per button):
import threading

def _watch_button(base, idx, callback):
    while True:
        base.buttons[idx].wait_for_value(1) # blocks until pressed
        callback(None)
        base.buttons[idx].wait_for_value(0) # wait for release before next press

for i, cb in enumerate([on_start, on_pause, on_left, on_right]):
    threading.Thread(target=_watch_button, args=(base, i, cb), daemon=True).start()
```


ISS-03 -- Face Angle Recognition (Medium)

Root cause: MediaPipe BlazeFace short-range model is optimised for frontal faces within ~2m. Rotation beyond ~20 degrees causes the face bounding box to shrink below the detection threshold at 640x480 resolution.

- Switch to BlazeFace full-range model (model_selection=1 in solutions API): handles wider angles and distances up to ~5m.
- If using Haar Cascade as the fallback, the alt2 cascade has better angle tolerance than the default frontal cascade.
- Add game guidance text prompting the player to face the camera directly as a UX mitigation.

ISS-04 -- Rapid Repeat on Hold (Low)

When a button is held, every poll cycle fires the callback, causing the face target to jump from 1 to 5 (or 5 to 1) instantly. This makes fine-grained adjustment difficult.

- Add a minimum repeat delay (e.g. 300ms) after the first edge before allowing repeated callbacks while the button remains held.
- Alternatively, require a release-then-press cycle: fire only on rising edge (addressed automatically by the interrupt-based fix for ISS-02).

3.3 Improvement Suggestions

#	Improvement	Expected Benefit
1	Replace MediaPipe with Haar Cascade on PYNQ Z2	FPS increase from 0.5 to ~25-30. Resolves ISS-01 and ISS-02.
2	Interrupt-based button handling via PYNQ wait_for_value()	Reliable tap detection regardless of frame rate. Resolves ISS-02 and
3	Detection frame skipping (every 3rd frame)	3x throughput improvement without changing detection backend.
4	Downscale input to 320x240 before detection	~20% faster inference; negligible accuracy loss.
5	Use BlazeFace full-range model (model_selection=1)	Better detection at angles and distances. Addresses ISS-03.
6	Add on-screen arrow indicators for button functions	Improved player guidance without relying on documentation.